A VHDL Version of the 32-Bit Stein GCD Core

Ahmed Saad Sweffi
Digital Design & Verification Engineer
RTL · FPGA · VLSI

***Abstract*—When I compared my two GCD designs, the core based on Stein's binary algorithm came out best overall. The original task asked for VHDL, so this short report presents a VHDL version of that core. It keeps the same ports, the same handshake and the same behavior in every clock cycle as the Verilog version. A self-checking testbench runs it on 75,555 test vectors, checks every result and every cycle count, and compares it cycle by cycle with the Verilog core. Everything passes:** $\gcd(14, 161) = 7$ **after 7 clock cycles, no case needs more than 64 cycles, and the two versions never differ.**

***Index Terms*—GCD, Stein's algorithm, VHDL, FPGA, simulation**

I. Introduction

The task asked for the VHDL code of an FPGA circuit that computes the greatest common divisor (GCD) of two 32-bit integers, with simulation results for $\gcd(14, 161)$ and with both processing time and circuit size in mind. I first wrote two designs in Verilog and compared them, one based on Stein's binary algorithm [1], [2] and one on Euclid's subtraction algorithm. The Stein core was the better one overall: it needs only 7 clock cycles for $\gcd(14, 161)$, never more than 64 cycles for any pair of 32-bit numbers, and does more work per logic cell. Since the task asked for VHDL, I also wrote this core in VHDL [3]. This report presents the code and checks it in simulation.

II. The VHDL Core

The core (Listing 1) has the same interface as the Verilog version: a clock, an active-low synchronous reset, a `start` pulse with the two operands, and the outputs `busy`, `done` and `gcd_out`. Inside, the registers a and b hold the operands and k counts the factors of two they have in common.

In every clock cycle the core applies one step of Stein's algorithm. When $a = b$ it is finished and returns a shifted left by k. When both numbers are even it halves both and increments k; when only one is even it halves that one; and when both are odd it replaces the larger number by half of the difference. Both differences, $a - b$ and $b - a$, are computed in parallel, and the borrow of $a - b$ chooses between them, so the core never waits for a comparison before it subtracts. If one operand is zero, the other one is the answer, and the core returns it in the start cycle. Every step removes at least one bit from a or b, which is why no 32-bit input needs more than 64 cycles.

Writing the core with the `unsigned` type of `numeric_std` keeps the code close to the hardware: `shift_right` is plain wiring, and the two subtractions map onto the carry chains of the FPGA. The listing leaves out the comment block at the top of the file.

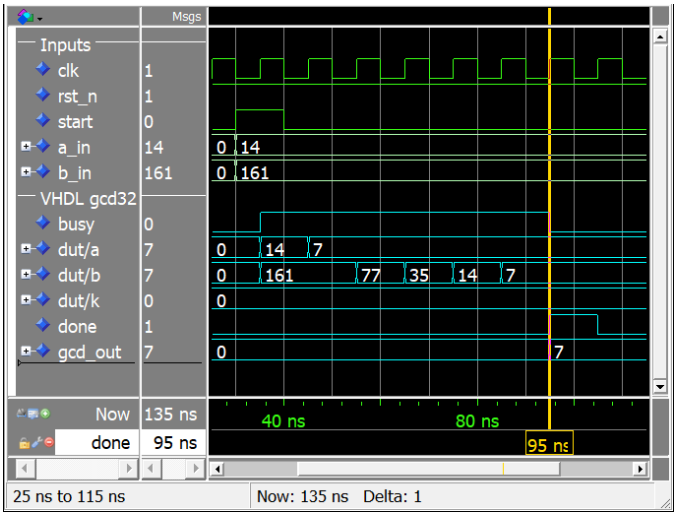


Fig. 1. Simulation waveform of the VHDL core for the required test, $\gcd(14, 161)$: `start` is sampled at 35 ns, the registers a, b and k show each Stein step, and the cursor marks `done` at 95 ns with `gcd_out` $= 7$, after 7 clock cycles.

III. Simulation

To check the VHDL core, I wrote a self-checking testbench in VHDL-2008. It drives the core with a 100 MHz clock and, for every operation, compares the result with a Euclid reference and the number of cycles with a cycle-by-cycle model of the algorithm. As an extra check, the testbench also runs the Verilog version of the core on the same inputs and compares the outputs of the two versions in every clock cycle. The test vectors are the required case $\gcd(14, 161)$, the 18 hand-picked cases used on the FPGA board (among them the worst case, $\gcd(2^{32} - 1, 2^{32} - 2)$), all 65,536 pairs of 8-bit operands, and the 10,000 random pairs of the board test.

Fig. 1 shows the required test. The `start` pulse is sampled at 35 ns. In the next cycles the core halves a from 14 to 7, reduces b from 161 to 77, 35 and 14 in three both-odd steps, and halves b to 7. At 95 ns a and b are equal, so the core raises `done` with `gcd_out` $= 7$, after **7 clock cycles**, or 70 ns at 100 MHz.

Listing 1. gcd32.vhd: Stein's binary GCD in VHDL

```vhdl
library ieee;
use ieee.std_logic_1164.all;
use ieee.numeric_std.all;

entity gcd32 is
    port (
        clk     : in  std_logic;
        rst_n   : in  std_logic;                      -- active-low sync reset
        start   : in  std_logic;
        a_in    : in  std_logic_vector(31 downto 0);
        b_in    : in  std_logic_vector(31 downto 0);
        busy    : out std_logic;
        done    : out std_logic;
        gcd_out : out std_logic_vector(31 downto 0)
    );
end entity gcd32;

architecture rtl of gcd32 is

    signal a, b      : unsigned(31 downto 0) := (others => '0');
    signal k         : unsigned(4 downto 0)  := (others => '0');   -- max 31
    signal busy_r    : std_logic := '0';
    signal done_r    : std_logic := '0';
    signal gcd_r     : unsigned(31 downto 0) := (others => '0');

    -- Two subtractors in parallel; the borrow of (a-b) selects the result,
    -- avoiding a serial compare->subtract path.
    signal a_minus_b : unsigned(32 downto 0);
    signal b_minus_a : unsigned(31 downto 0);
    signal a_lt_b    : std_logic;
    signal eq        : std_logic;

begin

    a_minus_b <= ('0' & a) - ('0' & b);
    b_minus_a <= b - a;
    a_lt_b    <= a_minus_b(32);                       -- borrow => a < b
    eq        <= '1' when a = b else '0';

    process (clk)
        variable sel : std_logic_vector(1 downto 0);
    begin
        if rising_edge(clk) then
            if rst_n = '0' then
                a      <= (others => '0');
                b      <= (others => '0');
                k      <= (others => '0');
                busy_r <= '0';
                done_r <= '0';
                gcd_r  <= (others => '0');
            else
                done_r <= '0';

                if start = '1' and busy_r = '0' then
                    -- Zero operands resolved immediately (no iterations)
                    if unsigned(a_in) = 0 or unsigned(b_in) = 0 then
                        gcd_r  <= unsigned(a_in) or unsigned(b_in);
                        done_r <= '1';
                    else
                        a      <= unsigned(a_in);
                        b      <= unsigned(b_in);
                        k      <= (others => '0');
                        busy_r <= '1';
                    end if;

                elsif busy_r = '1' then
                    if eq = '1' then
                        gcd_r  <= shift_left(a, to_integer(k)); -- restore 2^k
                        busy_r <= '0';
                        done_r <= '1';
                    else
                        sel := a(0) & b(0);
                        case sel is
                            when "00" =>                        -- both even
                                a <= shift_right(a, 1);
                                b <= shift_right(b, 1);
                                k <= k + 1;
                            when "01" =>                        -- a even
                                a <= shift_right(a, 1);
                            when "10" =>                        -- b even
                                b <= shift_right(b, 1);
                            when others =>                      -- both odd
                                if a_lt_b = '1' then
                                    b <= shift_right(b_minus_a, 1);
                                else
                                    a <= shift_right(a_minus_b(31 downto 0), 1);
                                end if;
                        end case;
                    end if;
                end if;
            end if;
        end if;
    end process;

    busy    <= busy_r;
    done    <= done_r;
    gcd_out <= std_logic_vector(gcd_r);

end architecture rtl;
```

TABLE I
SIMULATION RESULTS OF THE VHDL CORE

Test set	Vectors	Cycles, avg / max	Errors
$\gcd(14, 161)$	1	7 / 7	0
Hand-picked board cases	18	18.39 / 64	0
All 8-bit operand pairs	65,536	10.78 / 16	0
Random board pairs	10,000	35.76 / 58	0
Total	75,555		0
Cycles where VHDL and Verilog outputs differ			0

Table I sums up the results. All 75,555 operations give the right GCD with the expected number of cycles, and the VHDL and Verilog versions agree in every clock cycle. Even the cycle totals are exactly those of the Verilog design: 706,517 cycles for the 8-bit pairs and 357,608 cycles for the random board pairs.

IV. Conclusion

The VHDL version of the Stein core behaves exactly like the Verilog version, cycle for cycle, so its processing time is the same: 7 clock cycles for $\gcd(14, 161)$ and at most 64 cycles for any pair of 32-bit numbers. Since both versions describe the same registers and the same logic, the VHDL code should also synthesize to the same small circuit, although for this report I only simulated it.

References

[1] J. Stein, "Computational problems associated with Racah algebra," *J. Comput. Phys.*, vol. 1, no. 3, pp. 397–405, 1967.

[2] D. E. Knuth, *The Art of Computer Programming, Vol. 2: Seminumerical Algorithms*, 3rd ed. Reading, MA, USA: Addison-Wesley, 1997, Sec. 4.5.2.

[3] *IEEE Standard VHDL Language Reference Manual*, IEEE Std 1076-2008, 2009.